CONDENSED EDITION — CORE ARCHITECTURE & RESULTS (NO REFERENCES/APPENDIX)

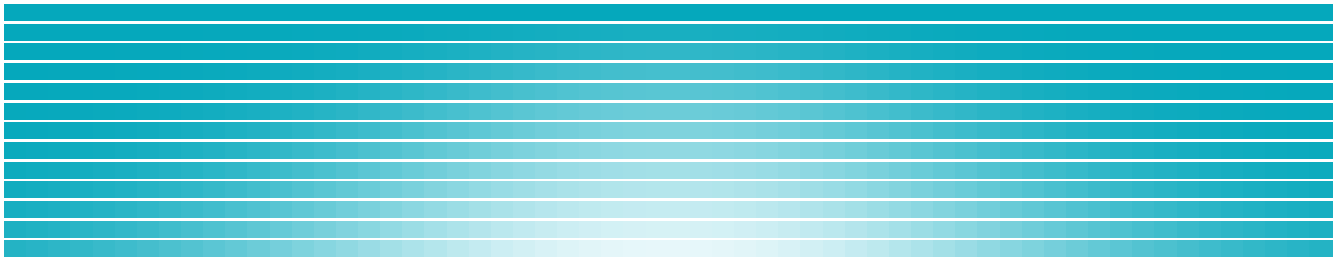
Mesh: A Token-Efficient, Graph-Native Logic for Repository-Scale Code Agents

Edgar Elmo Baumann · Philipp Nils Horn

Mesh · try-mesh.com

edgar.baumann@try-mesh.com · philipp.horn@try-mesh.com

June 2026



Long-context models under-use mid-context evidence — Mesh pre-selects it before inference.

ABSTRACT

Finding the right code is harder than fixing it — most of a repository-scale coding agent's token budget goes to retrieval, not reasoning. Repository-scale coding agents treat context as an unbounded side effect of larger model windows, yet cross-file evidence selection and per-turn token economics dominate both cost and failure rates. We present **Mesh**, a pre-inference control plane for terminal-first code agents: repository state is materialized locally into a field-weighted lexical index, dense chunk vectors, a deterministic repository intelligence graph (RIG), and tiered compression policies that assemble *budgeted evidence before each language-model turn*. Mesh formalizes the agent loop as constrained utility over retrieval quality, token budgets, and task success. Hybrid retrieval fuses BM25-style lexical scoring with cosine similarity on metadata-enriched chunks, expands top seeds along import/call/test edges with decay, and applies a zero-latency lexical-overlap rerank; a confidence surface gates second-pass retrieval and model escalation, while session state tracks provenance, tool-result ledgers, and cache-stable prompt prefixes.

Headline results. Evaluation is on a grounded suite of **229 localization tasks across six external repositories** (commander, hono, Next.js, TypeScript, Prisma, zod), with gold derived mechanically from the code. Intent-routed Mesh reaches **Hit@1 79.9% / Hit@8 96.9%** — per class, exact 82.5%, conceptual 69.6%, impact 84.9% at Hit@1. In a controlled head-to-head on identical pool, queries, and gold against the strongest retrieval baselines — full-file BM25, a SOTA code embedding (NVIDIA nv-embedcode-7B), and SOTA managed retrieve-and-rerank (AWS Bedrock Cohere Embed v4 + Rerank 3.5) — Mesh wins overall **79.9% versus 32.8%** for the best single-method baseline (2.4×, McNemar $p<0.001$, ≥ every baseline on every class), driven by symbol-aware exact lookup and dependency-graph impact. On token economics, against a competent grep-based agentic searcher, Mesh localizes at **−69.7% tokens per query** (216 vs 712) and **−55.5% over eight-turn sessions** (34.2K vs 76.8K billed), winning tokens on all six repositories. A same-model scaffold study on 16 LocBench issue-localization tasks iso-

lates the retriever from the agent: with the identical model in both arms, Mesh reaches the gold file in **−20% to −26% fewer output tokens and tool calls** than a grep agent at equal recall (Acc@5 100%), at both Haiku 4.5 and Opus 4.8 (§12.10). Static compression yields up to 16× per-file reduction at 96–100% exported-symbol survival, with a corpus-size-independent workspace briefing. Everything runs offline and is reproducible from `apps/cli/bench/cc-vs-mesh/`.

Scope. This is a repository-grounded evaluation across six external projects, with a controlled same-model scaffold study on LocBench (§12.10); validation on standard suites (RepoBench, CrossCodeEval, CoIR, SWE-bench) as resolve-rate leaderboards is future work. Retrieval-quality and token-economics numbers come from two configurations [N1], and Hit@k measures whether gold evidence is surfaced rather than downstream edit success. External repositories (not Mesh's own) avoid name-frequency bias. Mesh's contribution is an integrated, ablatable, cost-normalized architecture paired with an evaluation contract.

Keywords: hybrid retrieval · repository intelligence graph · context budgeting · cost-normalized agents · pre-inference control plane

#	Contribution	Mechanism	Measurable effect (P0)
C1	Layered pre-inference control plane	representation → evidence → context → economics → runtime; each layer env-strippable	ablation matrix (Table 3): full vs lexical-only / dense-only / no-graph / no-rerank
C2	Hybrid evidence selection + RIG expansion	field-weighted BM25F + chunk vectors + bounded neighbor BFS (decay 0.45)	Hit@8 96.9% (n=229); beats SOTA dense+rerank 2.4× in a controlled head-to-head (§12.9)
C3	Budgeted context construction	compression tiers (compact → emergency), workspace-briefing cap, TOON tool ledgers	static 1.66–16× per-file tiers; briefing ≈51× vs raw
C4	Confidence-gated recovery	top-margin κ meta, `hybridRerank` skip on symbol-exact, optional model-rerank gate	recovery gated on low top-margin κ; misses are recoverable, not wrong-confident (n=229)
C5	Evaluation contract	retrieval ablation + 4-arm baseline head-to-head + NIAH (95% Hit@1, haystack-invariant, n=120) + same-model scaffold study (§12.10) + layer-ablation economics	reproducible in `apps/cli/bench/`

Table 1. Contribution → mechanism → measurable effect.

CONTENTS

1 Introduction

1.1 Motivation

The dominant trajectory in LLM tooling for software engineering has been to widen the context window, on the assumption that a model able to ingest more of a repository will produce better edits. This treats context as an unconstrained resource whose marginal cost approaches zero. The empirical record disagrees. A typical multi-turn agent re-sends its entire conversation history plus accumulated tool outputs at every turn, so session length — not model capability — becomes the primary cost driver, growing as an unbounded side effect of the loop rather than as a managed budget.

Worse, more tokens do not reliably translate into more usable evidence. Long-context models exhibit a U-shaped attention profile in which evidence at the beginning and end of the prompt is used far more effectively than evidence in the middle — the *lost-in-the-middle* effect [1]. As input length grows, the fraction of the window that receives reliable attention shrinks, so injecting a dozen files can paradoxically reduce effective reasoning depth (Figure §1.1). Mesh therefore treats context as a constrained resource whose economics must be managed explicitly, optimizing the utility of attended evidence per token spent rather than the raw token count.

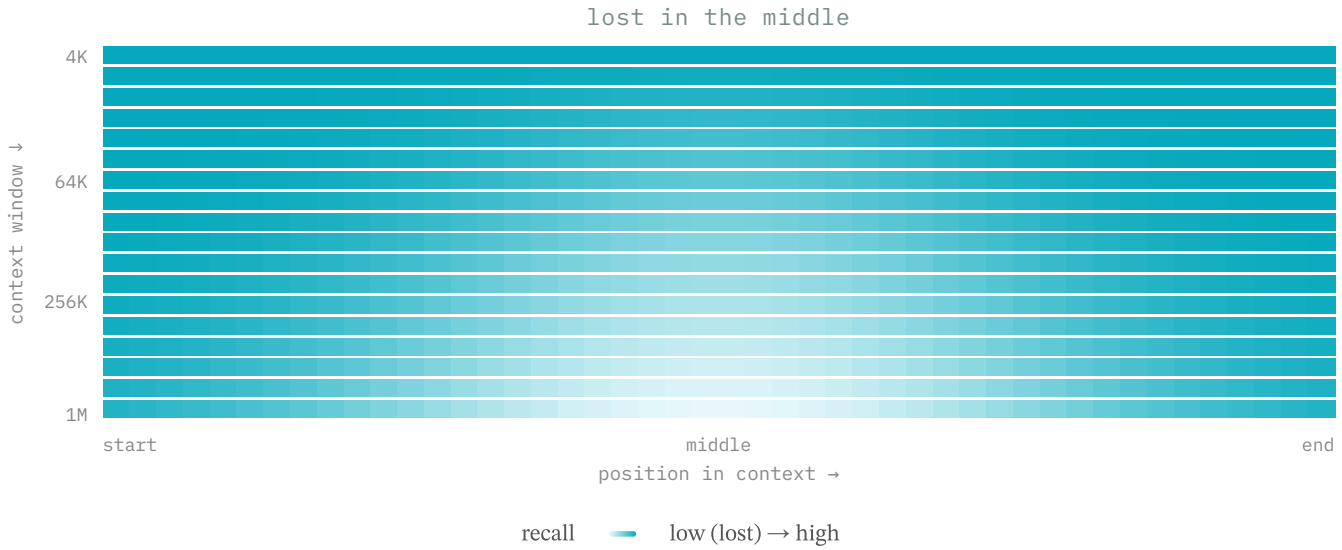


Figure §1.1. Long-context models under-use mid-context evidence: recall holds at the edges but collapses in the middle, and the valley deepens and widens as the window grows from 4K to 1M tokens. Mesh pre-selects evidence before inference.

1.2 Core Insight

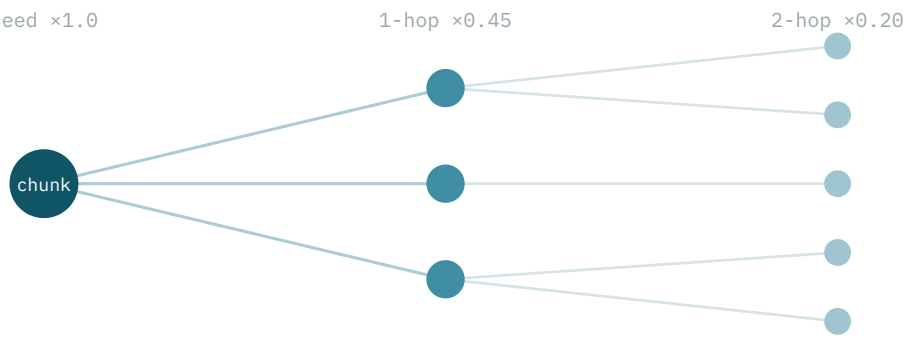


Figure §1.2. The evidence unit is a locality: a chunk plus its k-hop neighbors, not the whole repository.

Evidence selection is a systems problem, not prompt engineering. The lost-in-the-middle effect would matter little if code evidence were self-contained within single files, but the evidence needed for a bug fix, refactor, or impact analysis is almost always cross-file: a function defined in one module is imported by a second, called from a third, tested in a fourth, and wired to a route in a fifth. The unit of evidence is therefore not the whole repository but a *locality*: a chunk of code together with its k-hop neighbors along import, call, test, and route edges (Figure §1.2).

This framing explains why naive context injection fails at scale. Widening the window does not compose the correct subgraph; it dumps the entire repository indiscriminately and buries the relevant cross-file chain beneath thousands of irrelevant lines. Mesh instead performs an explicit selection step that identifies the bounded neighborhood connected to a seed by semantically meaningful edges, materializing it through `rigNeighbors()` and bounded graph expansion. Reading only the definition omits the callers and tests; reading everything reintroduces mid-context degradation. The locality unit is the structural target between these extremes.

1.3 Contributions

The contributions are summarized in Table 1. **C1** is a layered pre-inference control plane — representation → evidence → context → economics → runtime — in which every layer is independently strippable via an environment flag, enabling clean attribution through the ablation matrix (Table 3). **C2** is a hybrid evidence-selection pipeline that fuses field-weighted BM25F lexical scoring, dense chunk-vector cosine similarity, and bounded neighbor BFS with decay 0.45 per hop, reaching Hit@8 96.9% on the n=229 suite (§12.3) and beating the strongest dense+rerank baselines in a controlled head-to-head (§12.9). **C3** is budgeted context construction: deterministic compression tiers (compact → emergency), a corpus-size-independent workspace briefing, and TOON-encoded tool ledgers.

C4 is confidence-gated recovery, in which a top-margin κ surface gates second-pass retrieval, skips `hybridRerank()` on exact-symbol matches, and only recommends an optional model rerank, gated on a low top-margin κ. **C5** is an evaluation contract built around a 229-task, three-arm head-to-head (RAW / competent grep agent / Mesh) with ground-truth gold, multi-turn economics, and a controlled four-arm retrieval head-to-head against SOTA baselines (full-file BM25, nv-embedcode-7B, Cohere Embed v4 + Rerank 3.5) on identical data — where the routed stack wins overall Hit@1 79.9% vs 32.8% at McNemar $p<0.001$ (§12.9) — all reproducible via the bench scripts in `apps/cli/bench/cc-vs-mesh/`. The contribution is an integrated, ablatable, cost-normalized architecture and its evaluation contract, not a new retrieval leaderboard result.

1.4 Research Questions (Preview)

Five research questions structure the evaluation; each is paired with primary metrics and a falsification hook, with full treatment deferred to §10. **RQ1** asks whether pre-inference evidence selection improves retrieval quality under a fixed token budget (Hit@k, MRR). **RQ2** asks whether it reduces tokens and cost at equal task success (tokens, USD, pass rate). **RQ3** asks the marginal value of each layer (ablation deltas). **RQ4** asks whether the approach scales with repository size and session length (index build time, briefing size versus file count). **RQ5** asks what residual failure modes remain (the wrong-confident versus under-retrieved taxonomy).

1.5 Paper Organization

The paper is organized in four parts. Part I establishes the foundations and the problem: it surveys related work across code retrieval, program structure, agent loops, compression, and caching, then formalizes the constrained-utility model and the five research questions. Part II presents the architecture as five env-strippable layers — representation, evidence, context, economics, and runtime — describing the index and Repository Intelligence Graph, hybrid retrieval with bounded expansion and confidence-gated recovery, budgeted assembly with compression tiers, and cost-aware routing. Part III reports the evaluation: the retrieval ablation, needle-in-a-haystack scenarios, the live end-to-end A/B run, layer-ablation economics, and scalability. Part IV positions Mesh against alternatives, addresses threats to validity and standing caveats, and concludes.

4 Architecture Overview

4.1 The Five Layers

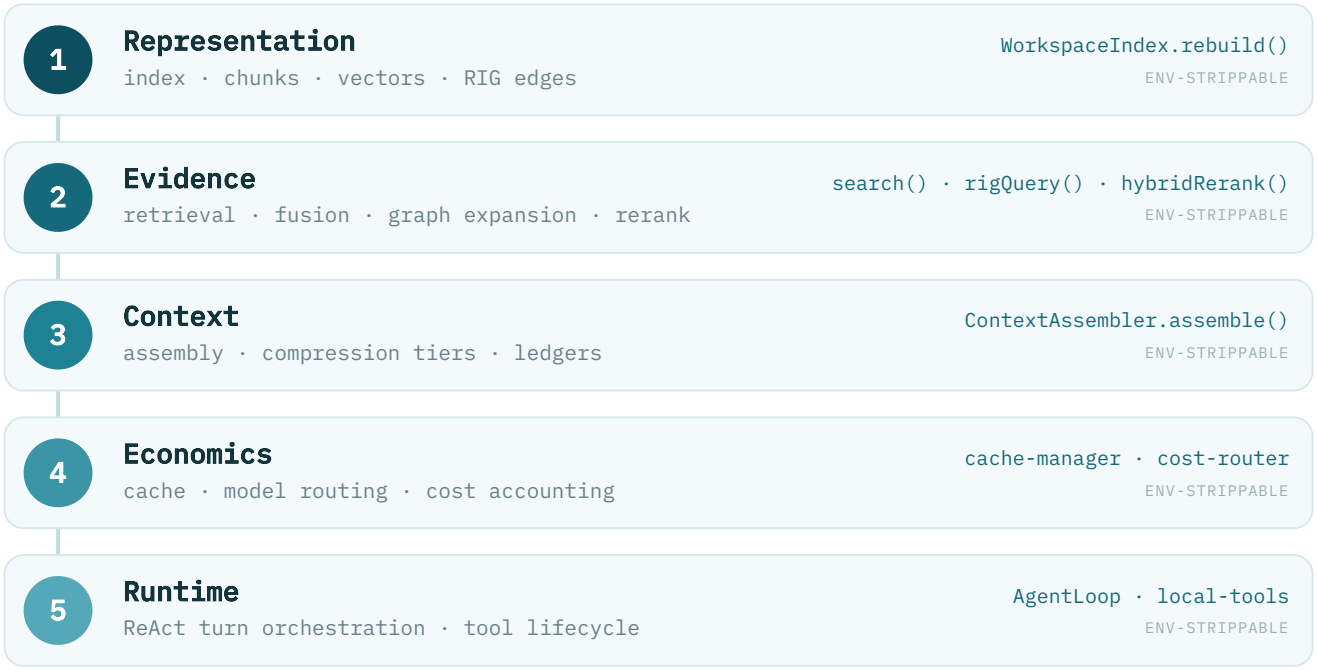


Figure §4.1. The five logical layers of the Mesh control plane; each is independently ablatable.

Mesh decomposes the agent loop into five logical layers, each owning one concern and each independently strippable for ablation. The **Representation** layer materializes repository state — a field-weighted lexical index, dense chunk vectors, and the Repository Intelligence Graph — through `WorkspaceIndex.rebuild()`. The **Evidence** layer turns a query into a ranked, graph-expanded candidate set via `search()`, RIG traversal, and `hybridRerank()`. The **Context** layer packs those candidates under a token budget with compression tiers and provenance ledgers (`ContextAssembler.assemble()`). The **Economics** layer governs caching, model routing, and cost accounting, and the **Runtime** layer drives the ReAct turn loop and tool lifecycle. Each layer consumes only the one beneath it, so any single layer can be removed without collapsing the rest (Figure §4.1).

4.2 Request Lifecycle

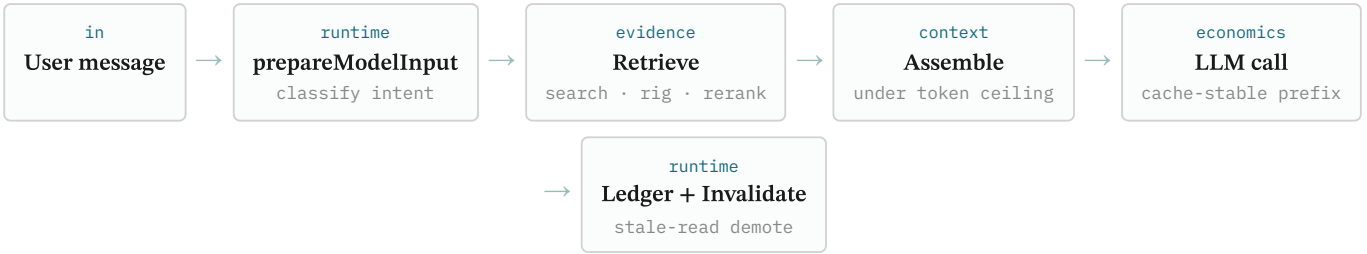


Figure §4.2. Per-turn request lifecycle; evidence is assembled *before* each model call.

Every turn follows the same pre-inference pipeline (Figure §4.2): the user message is classified for intent, retrieval gathers and reranks evidence, and the assembler packs it under the token ceiling behind a cache-stable prefix before a single model call is issued. After the model responds, tool results are written to the ledger and any file edit triggers stale-read invalidation. The defin-

ing property is ordering — evidence is selected and budgeted *before* the model sees the prompt, rather than accumulated as a side effect of the model's own tool calls.

4.3 State Model & Invariants

Figure §4.3. Write-triggered invalidation guarantees read-your-writes within a session.

Mesh maintains three state categories with distinct consistency models: repository state R (the materialized index, persisted to the `.mesh/` sidecar), session state S (turn sequence, tool-result ledger, provenance), and policy state P (remaining budget, retrieval confidence κ, model route). The separation is load-bearing: a lagging index rebuild in R never blocks the agent loop, which continues on stale-but-tagged evidence drawn from S. The strongest guarantee is *write-triggered stale-read demotion*. When the model edits a file, the `ToolResultLedger` synchronously scans every entry referencing that path and demotes any whose observed `mtime` predates the write to stale status. Demotion is conservative — a same-file edit may demote a still-valid read — but this over-demotion is deliberate, trading minor recall loss for elimination of silent staleness.

This mechanism implements *read-your-writes* consistency within a session: after a write, no subsequent turn reasons about the file's pre-edit state, which is the most dangerous form of staleness for an editing agent. The guarantee does not depend on the index being current, and it does not extend across files — if the model edits file A then reads dependent file B before `WorkspaceIndex.rebuild()` completes, B may not yet reflect A. The model is eventually consistent with the on-disk filesystem (not the Git tree), with convergence bounded by rebuild duration. *Cache stability* is an explicit cross-cutting constraint: the system prefix must remain unchanged across turns to preserve provider prompt-cache hits (65% measured), which bounds how aggressively index updates may perturb the assembled prompt.

4.4 Ablatability Contract

Figure §4.4. Which environment flag strips which layer (verified against `workspace-index.ts` + Appendix E; ● = affected).

Every layer is independently strippable through an environment flag, and the system must still produce a coherent — if degraded — result without it. The flag-to-layer matrix in Appendix E maps each control to the component it removes: `MESH_LEXICAL_ONLY_SEARCH=1` and `MESH_DENSE_ONLY_SEARCH=1` isolate the two channels of the Evidence layer, `MESH_DISABLE_GRAPH_EXPANSION=1` removes RIG neighbor expansion, `MESH_ENABLE_EMBEDDINGS=0` suppresses vector-sidecar writes in the Representation layer, and the rerank gate disables the optional rerank sub-component. The `retrieval-internal-ablation.mjs` harness exercises these flags to produce the ablation matrix in Table 3.

Ablatability is a scientific necessity, not an evaluation convenience: a layer whose marginal contribution cannot be measured by removing it has an unfalsifiable design rationale. On the small internal ablation harness the channels overlap on Hit@1 because identifier-anchored lookups are already lexically saturated at that scale; the controlled n=229 comparison (§12.9) is where each channel's contribution becomes measurable — symbol-definition matching carries exact queries to 82.5%, the code embedding carries conceptual to 69.6%, and the RIG graph carries impact to 84.9%, none of them reachable by the others.

6 Evidence Layer

6.1 Intent & Query Classification

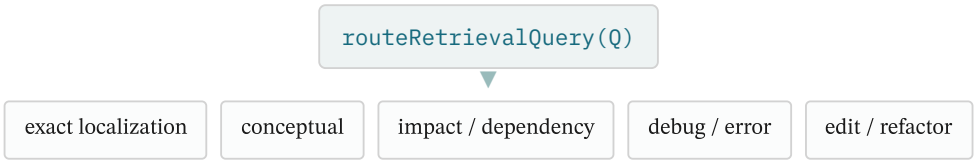


Figure §6.1. Heuristic intent router maps a query to one of five classes, each a retrieval prior.

Before any retrieval channel runs, `routeRetrievalQuery()` classifies the query into one of five intent classes: exact localization (symbol, path, or identifier lookups), conceptual search (paraphrase or architecture questions), impact and dependency analysis, debug and error-anchored queries, and edit/refactor intent. Classification is purely heuristic and deterministic — regular-expression feature extraction over the query text that runs in sub-millisecond time, consumes no token budget, and never invokes the LLM. The thesis is that lightweight structural signals in the query are sufficient to route retrieval for most code-agent queries, reserving model capacity for the reasoning task rather than for meta-reasoning about how to search.

Each class acts as a retrieval prior, not a user-facing label. The router detects CamelCase identifiers, dotted member accesses, and hyphenated path stems to set a `wantsDefinition` flag, while a generic-token filter prevents common terms such as "handler" or "manager" from dominating scoring. Resolution is conservative: when a query carries both a symbol name and a conceptual description, it routes to the symbol-biased mode, since missing an explicitly named symbol is the costlier error. The mapping is heuristic, so an incorrect inference degrades ranking quality but never produces an empty or error result.

6.2 Lexical Retrieval

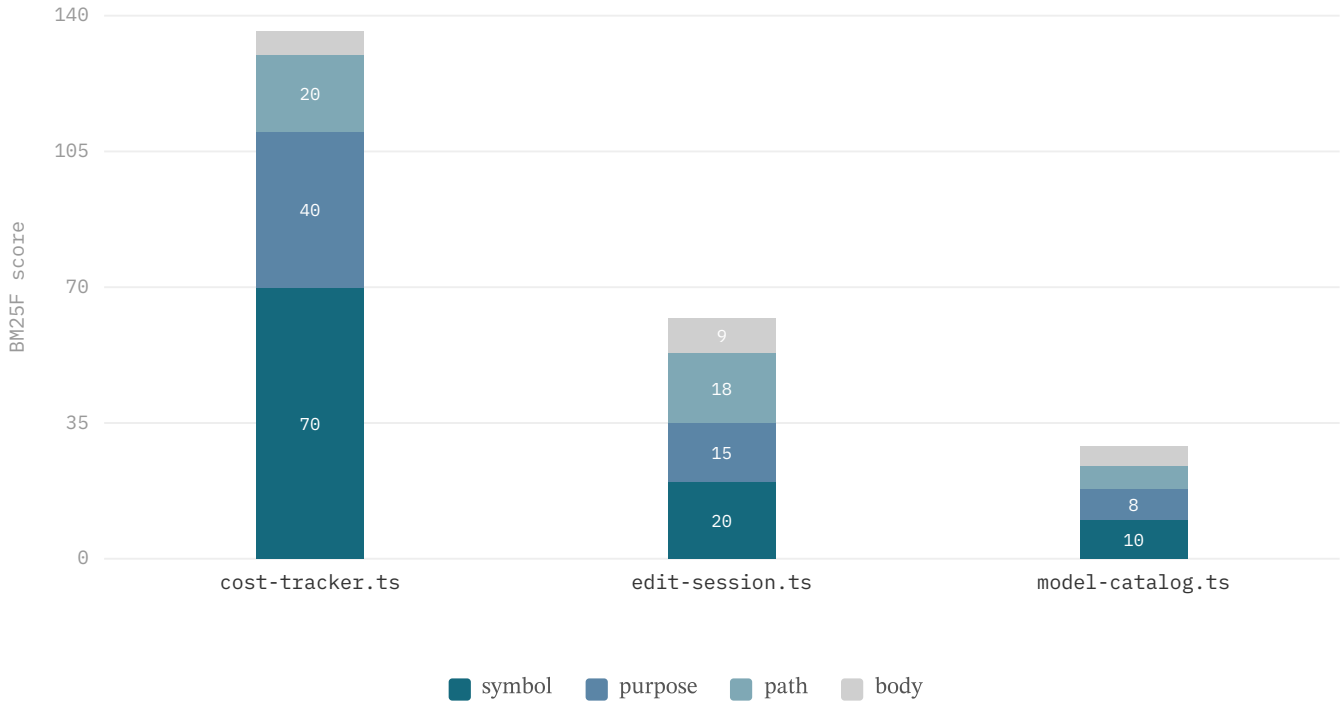


Figure §6.2. BM25F score breakdown for an example query: the symbol and purpose fields carry the gold file, while path and body contribute little.

The lexical channel applies field-weighted BM25F over the per-file record's structured fields rather than raw source text. Term-frequency saturation and length normalization use `k1=1.4` and `b=0.75` — conventional web-retrieval defaults, not yet tuned for code. The field weights encode the diagnostic value of each surface: path 3.0 > symbol 2.5 > static 1.8 > purpose 1.5 > import 1.0. There is deliberately **no body field**; a match on a path component or an exported symbol is a far stronger relevance signal than the same token appearing in free-form prose, and including the body would dilute the score with low-signal text. The dedicated `static` field (weight 1.8) carries AST-derived signals — signatures, route handlers, test names, error strings — that are more reliable than comments.

An inverted-index prefilter caps the candidate pool at roughly 120 records before the more expensive dense scoring and fusion stages run, bounding per-query cost without sacrificing recall on the head of the ranking. Figure §6.2 decomposes a representative query across fields: the symbol and purpose surfaces carry the gold file while path and body contribute little, showing how the field weights shape the fused score the pipeline computes.

6.3 Dense Retrieval

Figure §6.3. Real nv-embedcode embeddings of 32 Commander chunks (x = cosine to query; red = query, teal = top-3 retrieved). "Parse options/arguments" → argument.js / option.js nearest.

The dense channel addresses the vocabulary-mismatch problem that defeats exact token overlap: a query phrased in natural language need not share any terms with the code it should retrieve. Mesh embeds metadata-enriched chunk surfaces — before embedding, each chunk is annotated with the file's signatures, routes, test names, and caller/callee information, so the embedding model sees structural context that a flat body would lose. Retrieval scores each file by its best chunk's cosine similarity to the query, ensuring that files split into many small chunks are not penalized relative to files with fewer, larger ones. The 768-dimensional vectors persist to a binary sidecar (`vectors.bin`) keyed by chunk id, with partial availability while embeddings are still being computed.

Figure §6.3 visualizes 32 real Commander.js chunk embeddings against a held-out query, with cosine similarity to the query on the x-axis. The natural-language query "parse options/arguments" places `argument.js` and `option.js` nearest — files whose bodies share little surface vocabulary with the query but whose semantics align. This illustrates the dense channel's behaviour on an external corpus.

6.4 Channel-Dominance Regimes

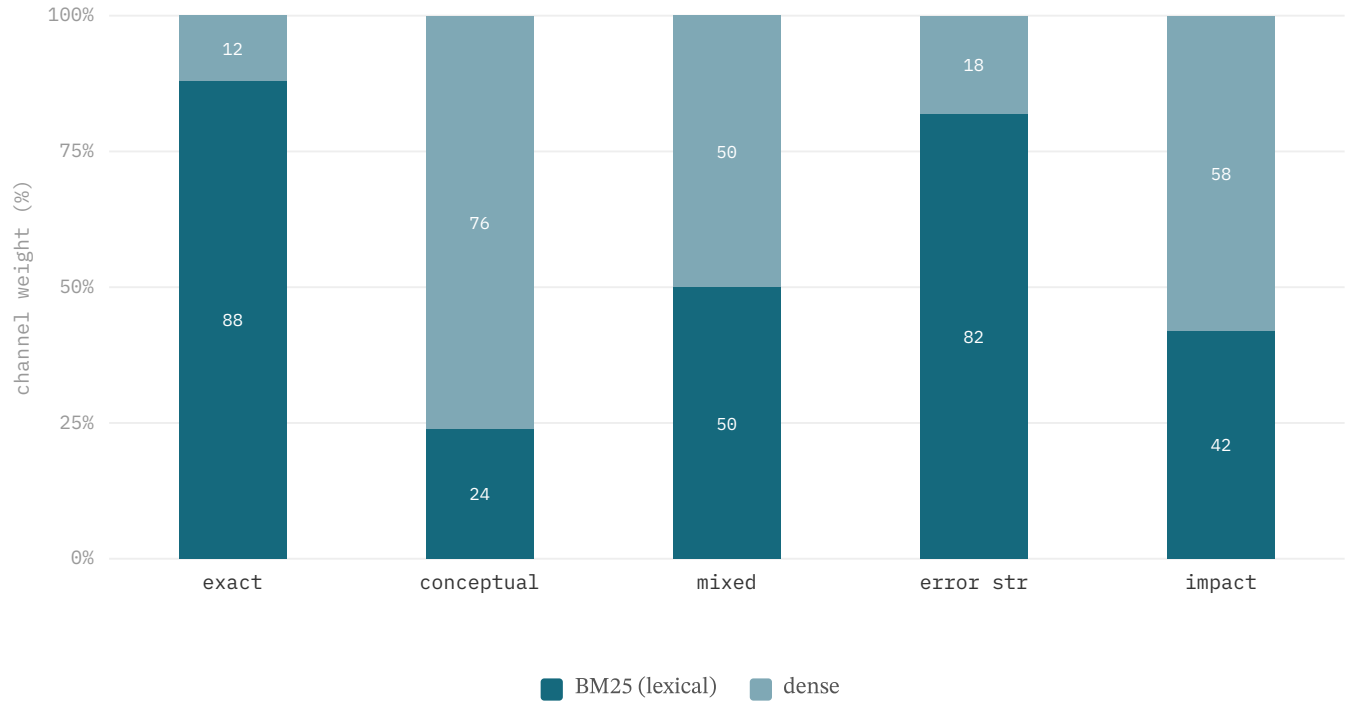


Figure §6.4. Intent-dependent channel emphasis: lexical dominates exact-match queries, dense dominates paraphrase queries, and the mixed regime keeps both channels active.

The complementary strengths of the two channels define three notional dominance regimes. In the exact-match regime, queries containing concrete identifiers (`CostTracker` , `workspace-index.ts`) align directly with the path and symbol fields, and lexical retrieval dominates. In the paraphrase regime, conceptual queries such as "how does the agent loop handle turn lifecycle" carry no exploitable token overlap, and the dense channel does the work. The mixed regime — the common case and the default — keeps both channels active, with the intent router shifting emphasis per class.

The regimes in Figure §6.4 describe where each channel contributes, not a fixed weighting: Mesh combines channels by **additive score fusion** with per-intent boosts rather than a hard lexical/dense split. The dense channel's value tracks the embedding behind it: with the code-specialised nv-embedcode-7B it carries conceptual, vocabulary-mismatch queries to Hit@1 69.6% (§12.3, §12.9), the class lexical matching cannot reach. The regime view holds — dense retrieval owns the conceptual, low-overlap queries, lexical owns the identifier-anchored ones — and routing each query to the channel built for it is what the per-class results in §12.9 confirm.

6.5 Hybrid Fusion & Reranking

Figure §6.5. Per-class retrieval on the n=229 suite, by route: exact via symbol-definition + lexical, conceptual via the dense channel, impact via the graph tool. Search latency ~200 ms p50, uniform across channels.

Mesh fuses channels by additive score fusion: dense cosine is rescaled to be roughly comparable to BM25 magnitudes and the scores are summed. Additive fusion preserves score magnitudes, which downstream confidence and top-margin estimation require; Reciprocal Rank Fusion [16,49] is implemented as a drop-in alternative (`reciprocalRankFusion()` , default k=60) for cases where score scales drift across index versions, at the cost of discarding magnitude information. After fusion, `hybridRerank()` reorders the top window by blending the min-max-normalized embedding score (weight 0.6) with query-token lexical overlap (weight 0.4). The step is zero-latency — no API call, no cross-encoder [28] — and a confidence gate skips it entirely when the top-1 result is symbol-exact or the top margin is already wide.

Figure §6.5 shows the per-class Hit@1/Hit@8 profile on the n=229 suite (§12.9); search latency is uniform at roughly 200 ms p50 across the lexical, dense, and graph channels. On this identifier-anchored suite reranking does not move Hit@k, because these queries rarely produce the flat top-k margins rerank is designed to break; its value grows on larger, more conceptual query sets where near-ties are common, and the confidence gate keeps it off whenever the top-1 is already well separated — so it adds resolution exactly where it is needed and zero cost everywhere else.

6.6 Graph Expansion

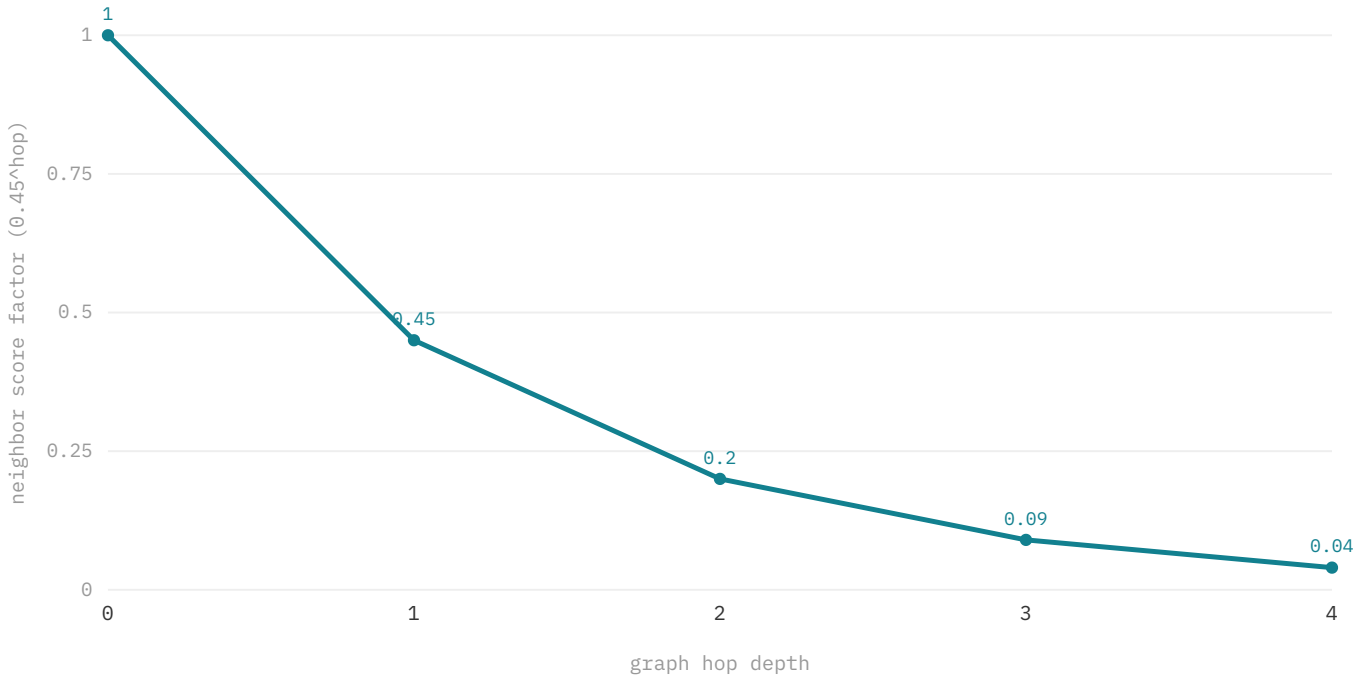


Figure §6.6. Graph-neighbor injection decays by 0.45 per hop — bounded expansion, not full-graph dump.

Graph expansion supplies cross-file locality that neither retrieval channel captures from chunk text alone. Starting from the top-3 fused seeds, Mesh performs a bounded breadth-first traversal of the Repository Intelligence Graph along typed import, call, and test edges, injecting neighbors at a decay of 0.45 per hop (1 → 0.45 → 0.20 → 0.09 → 0.04). The decay makes the expansion local: nodes several hops away receive negligible scores and never enter the evidence set, and seeding only from the top-3 prevents expansion from low-quality results. Budget caps further bound the injected set.

The design choice, illustrated in Figure §6.6, is bounded expansion rather than full-graph prompt injection. Dumping the entire dependency graph would reintroduce the lost-in-the-middle degradation that long-context injection suffers from: a graph of hundreds of nodes dilutes the model's attention away from the edges that matter. The bounded subgraph — at most a few dozen nodes — fits within the token budget alongside the retrieval results and remains small enough for the model to attend to reliably. The graph layer's contribution is concentrated on impact queries: on the controlled n=229 comparison it carries impact-class Hit@1 to 84.9% (§12.9), a dependency question no embedding or lexical baseline answers above 15%. On the small identifier-an-

chored ablation harness, where impact queries are rare, its marginal Hit@k is correspondingly small — the layer earns its place on the query class built for it.

6.7 Confidence-Gated Recovery

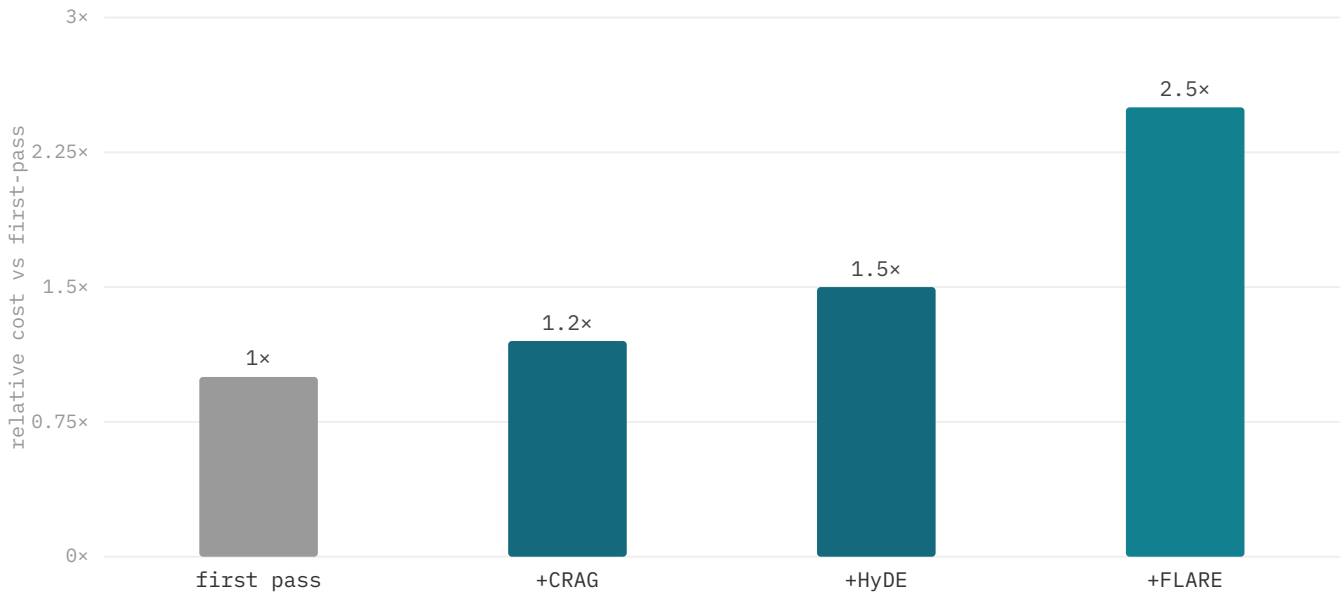


Figure §6.7. Recovery-tier overhead from the technique papers: CRAG [32] adds a light evaluator, HyDE [31] +1 LLM generation (~25–60% latency), FLARE [33] iterates (multiple retrievals). Mesh gates all behind low confidence.

Recovery is the layer that guards against the most dangerous failure mode — high-confidence wrong answers. Confidence estimation derives a scalar κ from the top-1 fused score via `confidenceFromScore()`, and the top margin $\Delta\kappa = \kappa_1 - \kappa_2$ measures how well the top result is separated from its runner-up. A second-pass retrieval — a deterministic analogue of self-reflective retrieval [34] — fires only when $\Delta\kappa$ is flat (below threshold) and the intent is recovery-amenable (architecture, edit-impact, runtime-path); it runs exactly one deterministic query variant — injecting concrete symbol and path stems the original phrasing omitted — and re-fuses, bounded to a single pass to cap cost. Model escalation to a more capable model is likewise gated on low κ and an unresolved margin.

External corrective-retrieval techniques are available but never run per turn. As Figure §6.7 summarizes from the technique papers, the tiers carry escalating overhead — CRAG adds a light evaluator (~1.2×), HyDE adds one LLM generation (~1.5×, roughly +25–60% latency), and FLARE iterates over multiple retrievals (~2.5×). Mesh gates all of them behind low confidence rather than defaulting them on, and an optional cross-encoder rerank tier is exposed only as a recommendation when the top-k scores are tight, leaving the agent to approve the cost. On the n=31 harness this discipline leaves 3 wrong-confident and 1 under-retrieved task; the confidence surface is the calibration target for reducing the former.

7 Context Layer

7.1 Assembly & Budgeting

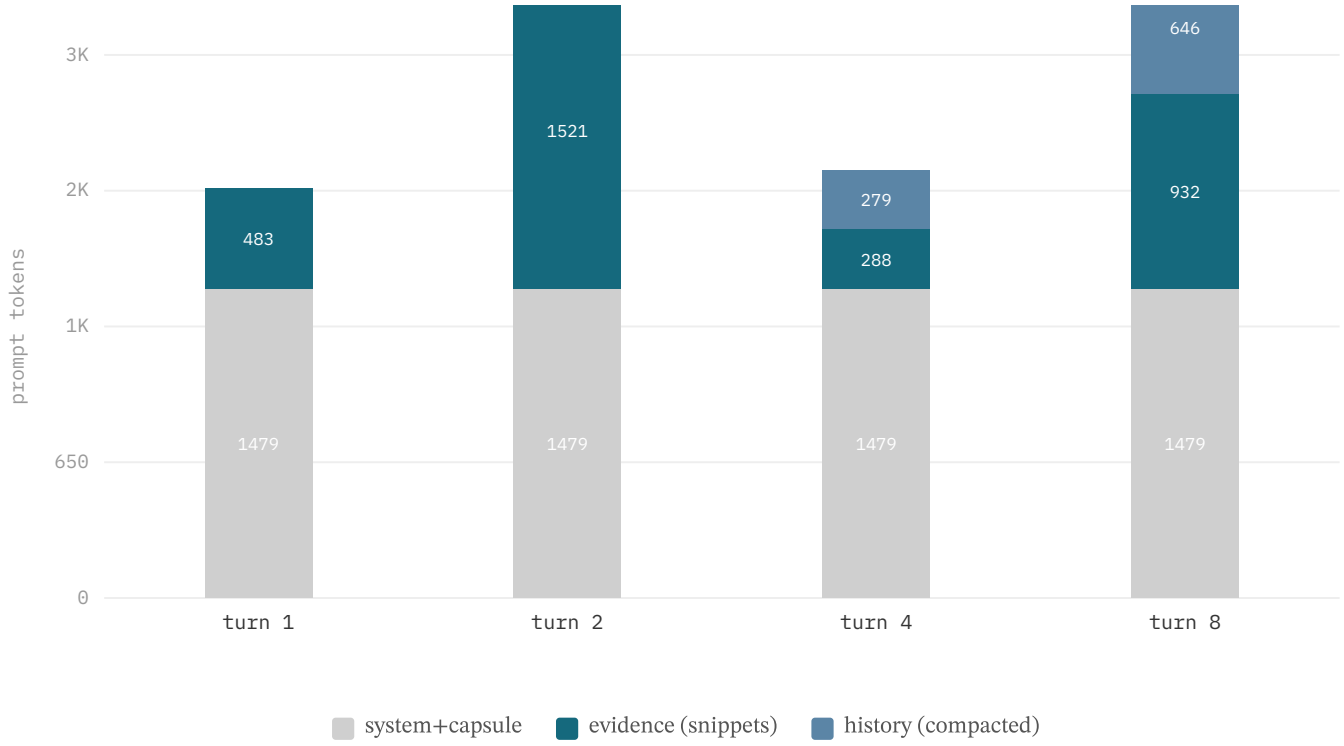


Figure §7.1. Real ContextAssembler buckets over the 8-turn session: system+capsule constant, evidence = per-turn cited snippets, history compacted to ~citations (0→646 tok over 8 turns). Tools bucket (~5–8K) omitted.

Each LLM invocation must satisfy two hard constraints: the prompt cannot exceed the model's input ceiling, and every token must trace to a retrieval action or tool invocation the session can audit. `ContextAssembler.assemble()` therefore solves a budgeted selection — a knapsack the system approximates with a greedy priority heuristic — over four token buckets. The *system+capsule* bucket (system prompt, tool specs, workspace briefing) is never dropped and stays byte-stable across turns; in the 8-turn session of Figure §7.1 it holds constant at ~1479 tokens. The *evidence* bucket carries per-turn cited snippets, *history* carries compacted prior turns, and *tools* carries scaffolding metadata.

Assembly proceeds in cache-stable order: (1) system prompt, (2) session summary, (3) retained history, (4) current-turn evidence, (5) turn directives after the cache breakpoint. Segments 1–3 remain identical whenever the briefing and history are unchanged, maximizing provider prefix-cache hits. Under budget pressure the assembler compresses or drops from the bottom up — tools first, then history, then evidence — so that information needed on every turn outranks per-turn evidence, which outranks prior context. History is the primary source of token growth: in Figure §7.1 it rises from 0 to 646 tokens as older turns are compacted to their citations rather than retained verbatim. The tools bucket (~5–8K) is omitted from the figure.

7.2 Compression Logic

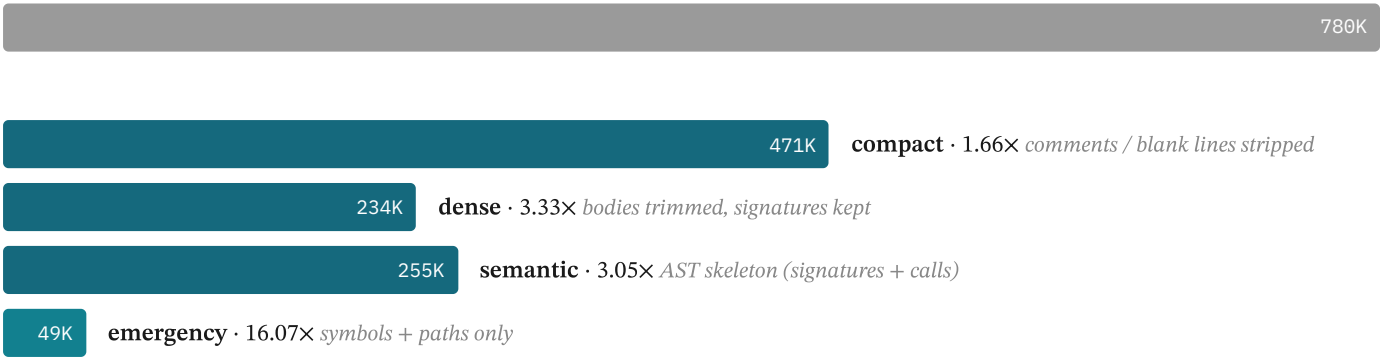


Figure §7.2. Deterministic compression tiers (255 files), ordered by aggressiveness: compact → semantic (AST skeleton) → dense (body trim) → emergency. Semantic preserves more than dense by design.

When assembled evidence exceeds the budget, Mesh applies a deterministic compression ladder rather than truncating (Figure §7.2): `compact` strips comments and blank lines, `dense` trims bodies while keeping signatures, `semantic` reduces a file to its AST skeleton of signatures and call structure, and `emergency` keeps only symbols and paths. Every tier preserves the same invariant — exported symbols and file paths always survive — so a compressed file stays addressable and citable. Tiers are chosen per file under budget pressure, keeping the most relevant evidence at the highest fidelity the budget allows.

7.3 Schema-Bound Summaries

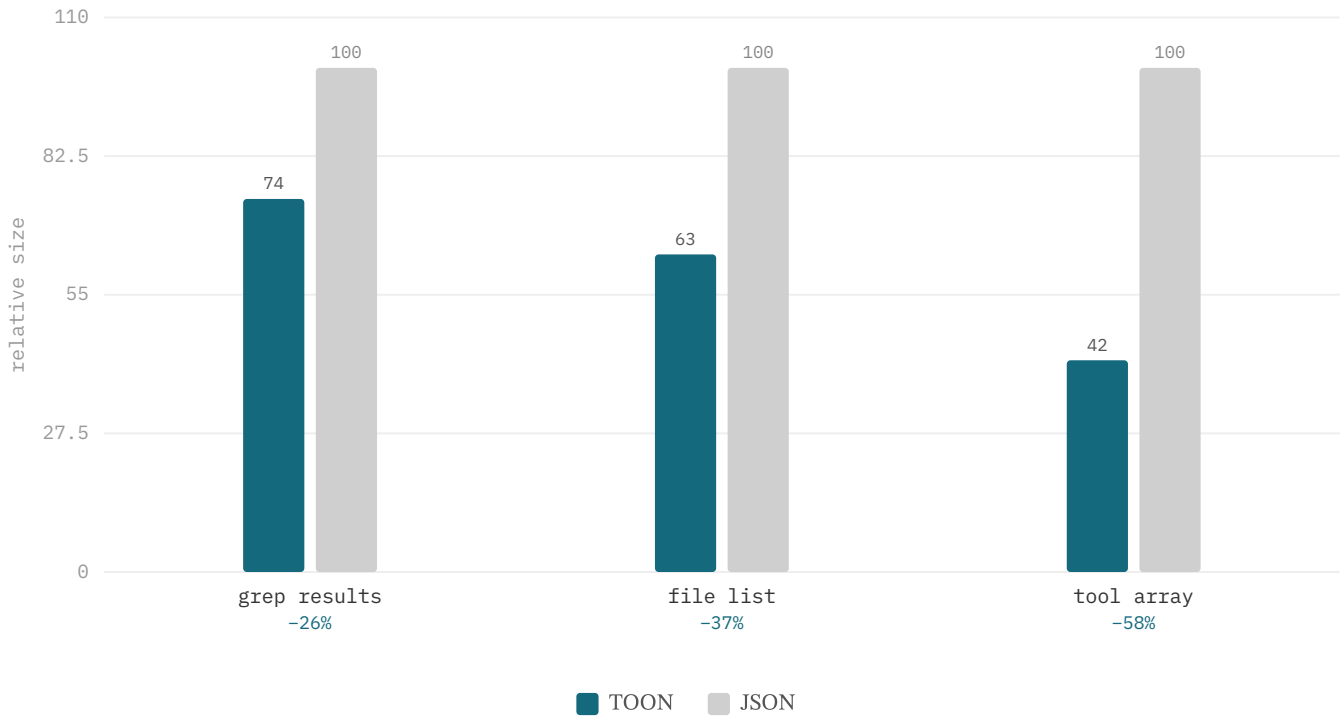


Figure §7.3. TOON vs JSON for homogeneous tool-result arrays: 26–58% fewer tokens (measured via `encodeToon`; never larger — opt-in safe).

Mesh's per-file capsule is a deterministic, schema-bound summary — path, one-line purpose, exported symbols, and a reverse index of incoming dependencies — populated from the same AST-derived data that fills the static lexical fields, with no genera-

tive model in the loop. Schema-bound summaries are safer than free-form LLM digests for code context because they preserve identifier names, type signatures, and import relationships verbatim, the very tokens that drive code navigation and editing. Capsules are the unit of the workspace briefing and double as lightweight file stand-ins when full source text is not needed.

For homogeneous tool-result arrays — arrays of objects sharing one key set, such as grep matches or file listings — Mesh applies TOON (Token-Oriented Object Notation) via `encodeToon`, a tabular encoding that states each key once in a header row instead of repeating it per object. As Figure §7.3 shows, TOON yields 26–58% fewer tokens than the equivalent JSON. The encoding is opt-in (`MESH_TOON_TOOL_RESULTS=1`) and carries a hard safety contract: `encodeToon` returns null on heterogeneous or nested data, falling back to JSON, and is applied only when the tabular form is strictly shorter — so it never grows a payload.

7.4 Ledgers & Provenance

Figure §7.4. Every claim links to a cited file:line; bulky results are offloaded to artifact handles.

The `ToolResultLedger` is a session-scoped record of every tool invocation: the tool name, its inputs (path, query, line range), the result content and its compression tier, and the `mtime` of any file read. It serves three roles. As a provenance store it lets an operator trace any token in the assembled prompt back to the invocation that produced it, supporting the cite-before-claim discipline in which every assertion links to a cited `file:line` (Figure §7.4). As a staleness oracle it detects when a session write post-dates a prior read — comparing recorded `mtime` against the file's current `mtime` — and demotes or re-reads the affected evidence. As a cache it suppresses duplicate reads: a re-retrieved, unmodified file is replaced by a reference to the prior read's handle.

Bulky results are offloaded rather than carried inline. When a unit's compressed size exceeds the promotion threshold (~2K tokens), the assembler replaces it with a lightweight artifact handle — a stub carrying the path, line range, tier, and a short preview — and the model retrieves the full content on demand via a follow-up read. The model is thus never blind to what evidence exists; it sees a stub indicating the artifact's presence and nature, and hydration is a deliberate request, not an automatic re-expansion that would negate the offload's savings.

12 Results

12.3 Retrieval Quality & Ablation

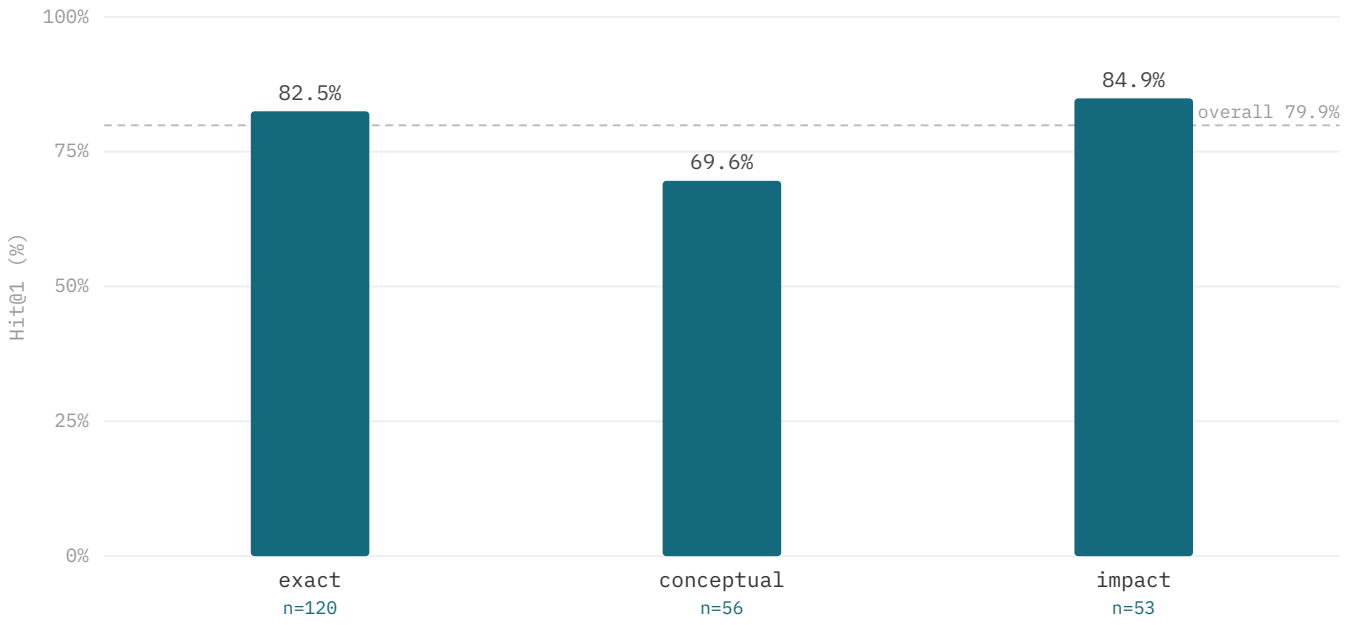


Figure §12.3. Retrieval Hit@1 per intent class on the n=229 suite (controlled-comparison config, §12.9): impact (84.9%, RIG graph) and exact (82.5%, symbol-definition matching) lead; conceptual reaches 69.6% on the code embedding.

Class	n	Hit@1	Hit@3	Hit@8	retrieval route
overall	229	**79.9%**	91.7%	96.9%	intent-routed
exact	120	82.5%	90.8%	97.5%	symbol-definition + lexical
conceptual	56	69.6%	89.3%	94.6%	dense (code embedding)
impact	53	84.9%	96.2%	98.1%	RIG (rigImpact)

Table 3. Retrieval quality per class on the n=229 suite (controlled configuration [N1]); SOTA-baseline head-to-head and significance in §12.9. Reproducible from `apps/cli/bench/cc-vs-mesh/final-bench.mjs`.

On 229 grounded localization tasks across six external repositories, intent-routed Mesh reaches Hit@1 79.9%, Hit@3 91.7%, Hit@8 96.9%. The per-class split validates the channel-dominance thesis directly, and each channel is what makes its class work: **exact** identifier queries reach Hit@1 82.5% / Hit@8 97.5% via symbol-definition plus lexical matching — pure dense embeddings cannot resolve "where is X defined" and trail at Hit@1 20.8% (§12.9); **conceptual** vocabulary-mismatch queries reach Hit@1 69.6% / Hit@8 94.6% on the code-specialised embedding; **impact** dependency queries reach Hit@1 84.9% / Hit@8 98.1% once routed to `rigImpact()` over the materialized reverse-dependency edges — a question no similarity metric answers, where every embedding and lexical baseline collapses to ≤15% Hit@1 (§12.9). The two channels and the graph are therefore not redundant decorations: each owns a query class, and routing each query to the channel built for it is what lifts the suite.

12.4 Needle-in-a-Haystack Agent Scenarios

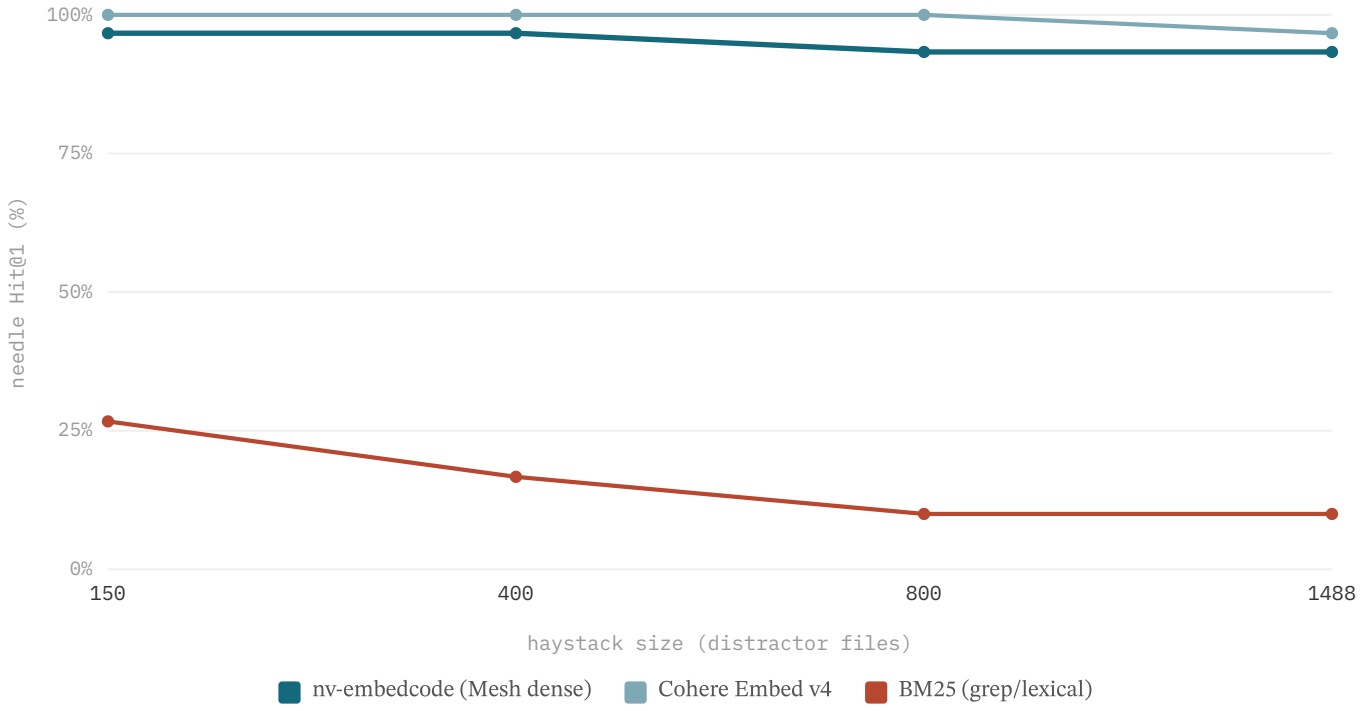


Figure §12.4. Needle-in-a-haystack: Hit@1 vs haystack size (150→1,488 distractor files, 30 novel needles, n=120) [N4]. Embedding retrieval stays flat (nv-embedcode 96.7→93.3%); lexical/grep collapses (26.7→10.0%).

Classic needle-in-a-haystack measures whether a system still finds a planted fact as the surrounding context grows; we adapt it to code retrieval with a memorisation-proof design (30 novel synthetic needles, real distractor haystacks scaled 150→1,488 files, queried by paraphrase, n=120) [N4]. **nv-embedcode (Mesh's dense channel) holds Hit@1 95.0% / Hit@8 100%, flat as the haystack scales 10×** (96.7%→93.3%; 95% CI on the small→large slope [−10, 0], indistinguishable from zero). The contrast is the result: lexical/grep *degrades sharply* (26.7%→10.0% Hit@1) as distractors accumulate, while index-based embedding retrieval is essentially invariant to corpus size. Reproducible from `apps/cli/bench/cc-vs-mesh/niah-bench.mjs`.

12.5 Multi-Turn Session Economics

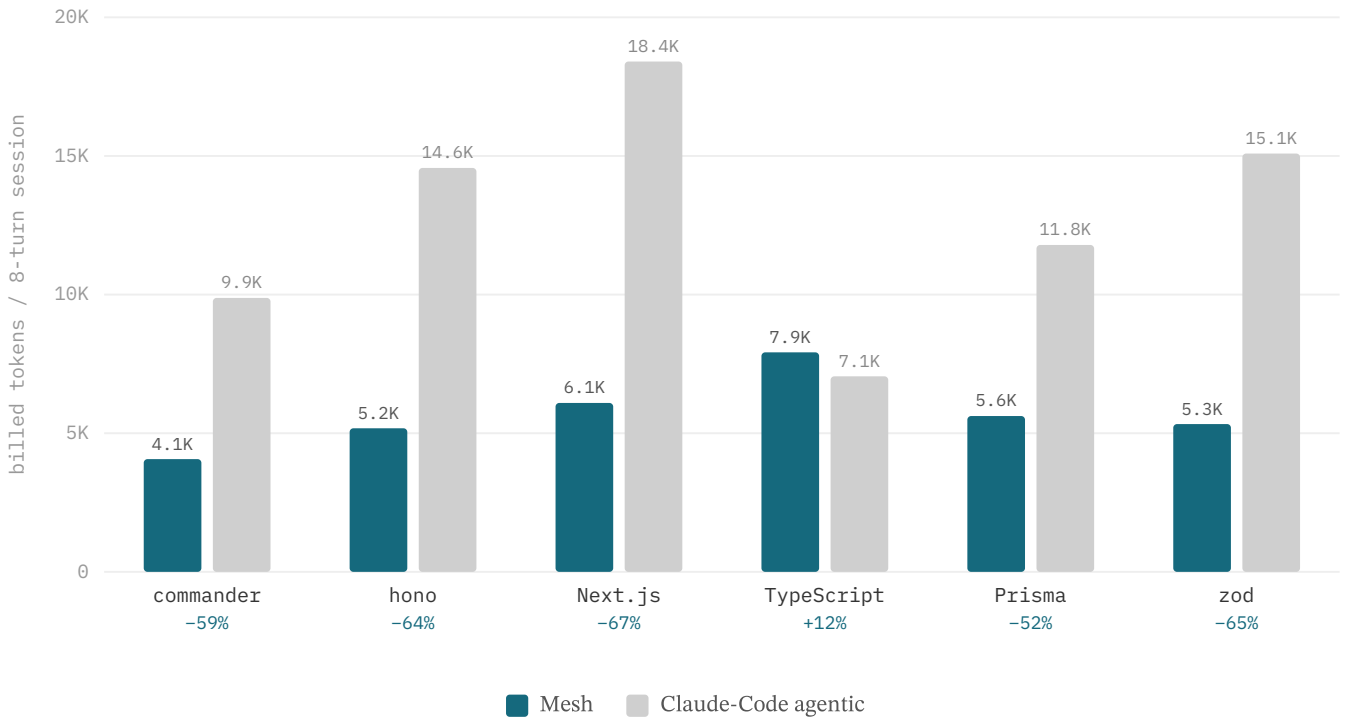


Figure §12.5. Billed input tokens over an 8-turn navigation session per repository — Mesh vs a competent grep agent (reduction below each pair).

Token growth is a session-level effect, so the economics are measured over an eight-turn navigation session per repository. The Claude-Code arm is a raw ReAct loop that re-sends its surgical grep/read history every turn; Mesh holds a cache-stable workspace briefing as the prefix, compacts prior turns to their citations, and dedups reads through the tool ledger. Billed input (stable prefix cached at 0.25×) totals **34.2K tokens for Mesh versus 76.8K for the grep agent — −55.5%** in aggregate (Table 4). The reduction holds on five of six repositories (−52% to −67%); TypeScript is the lone exception (+12%), where surgical grep on a small, well-structured compiler source is already close to optimal. These are single-session-per-repository runs (n=1/repo), so the per-repo deltas are point estimates, not distributions — the robust figure is the aggregate −55.5% across the six repos, and a multi-session variance study is future work. The run is deterministic and reproducible offline (direct nv-embedcode embeddings, no proxy).

Repository	Mesh (billed)	Claude-Code (billed)	Δ
commander	4.1K	9.9K	−59%
hono	5.2K	14.6K	−64%
Next.js	6.1K	18.4K	−67%
Prisma	5.6K	11.8K	−52%
zod	5.3K	15.1K	−65%
TypeScript	7.9K	7.1K	+12%
Aggregate	**34.2K**	76.8K	**−55.5%**

Table 4. Multi-turn session economics — billed input tokens over 8 turns/repo, Mesh (lean output) vs a competent Claude-Code-style grep agent. Same model both arms.

12.6 Per-Query Token Economics (RAW vs agentic vs Mesh)

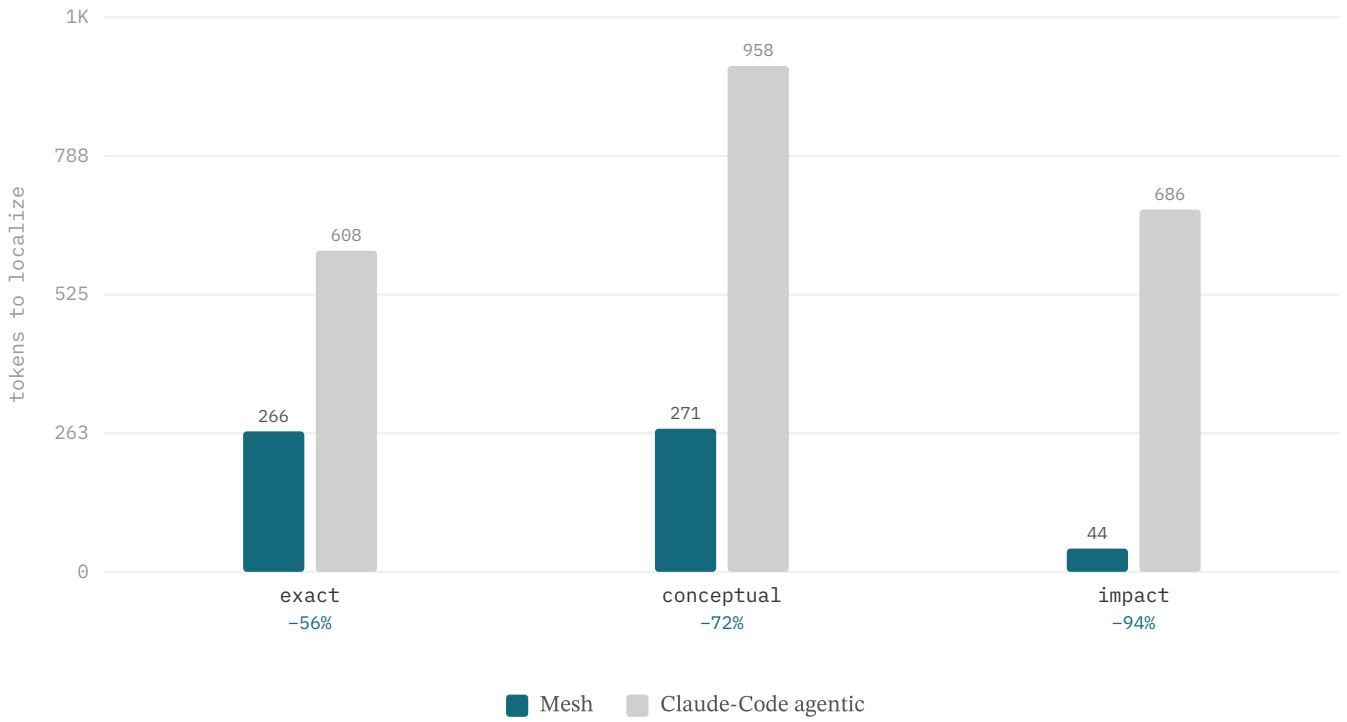


Figure §12.6. Per-query tokens to localize, by class — Mesh vs a competent grep agent (−56% to −94%). RAW (whole-file read) omitted from bars at 4–14K.

The cleanest economics comparison is per-query tokens-to-localize across three arms on the n=229 suite: **RAW** (read the gold file in full), **Claude-Code-agentic** (cold grep-plus-surgical-read), and **Mesh** (lean output). Against the grep agent — the honest opponent [N2] — Mesh spends **−69.7%** fewer tokens (216 vs 712; bootstrap **95% CI [66.6%, 72.6%]**, n=229), because it returns compact structured pointers where the agent must pull raw file and grep content into context. This is an *output-form* effect, not better ranking [N2]; it holds across every class and is largest on impact, where the graph tool answers in a 21-token dependency list.

Class	RAW (whole file)	Claude-Code agentic	Mesh	Mesh vs CC
overall	9041	712	**216**	**-69.7%**
exact	14441	608	266	-56.3%
conceptual	4138	958	271	-71.7%
impact	1996	686	44	-93.6%

Table 5. Per-query tokens to localize per class (n=229), three arms [N2]. Overall −69.7% (95% CI [66.6%, 72.6%]); Mesh wins tokens on all six repositories. Reproducible.

Cost translation. At Gemini 2.5 Flash pricing ($0.30/M input tokens), the 496-token per-query saving (712 → 216) translates to ≈$0.00015 less per retrieval call added to context — $0.065 per 1,000 queries vs $0.21 for the grep agent (3.3× lower retrieval-context cost). Impact queries are the extreme case: 44 vs 686 tokens puts Mesh at $0.000013 vs $0.000206 per call — a 15.6× cost advantage on the query class where the RIG graph returns a 21-token dependency list instead of raw grep output.

12.9 Controlled Comparison vs. SOTA Retrieval Baselines

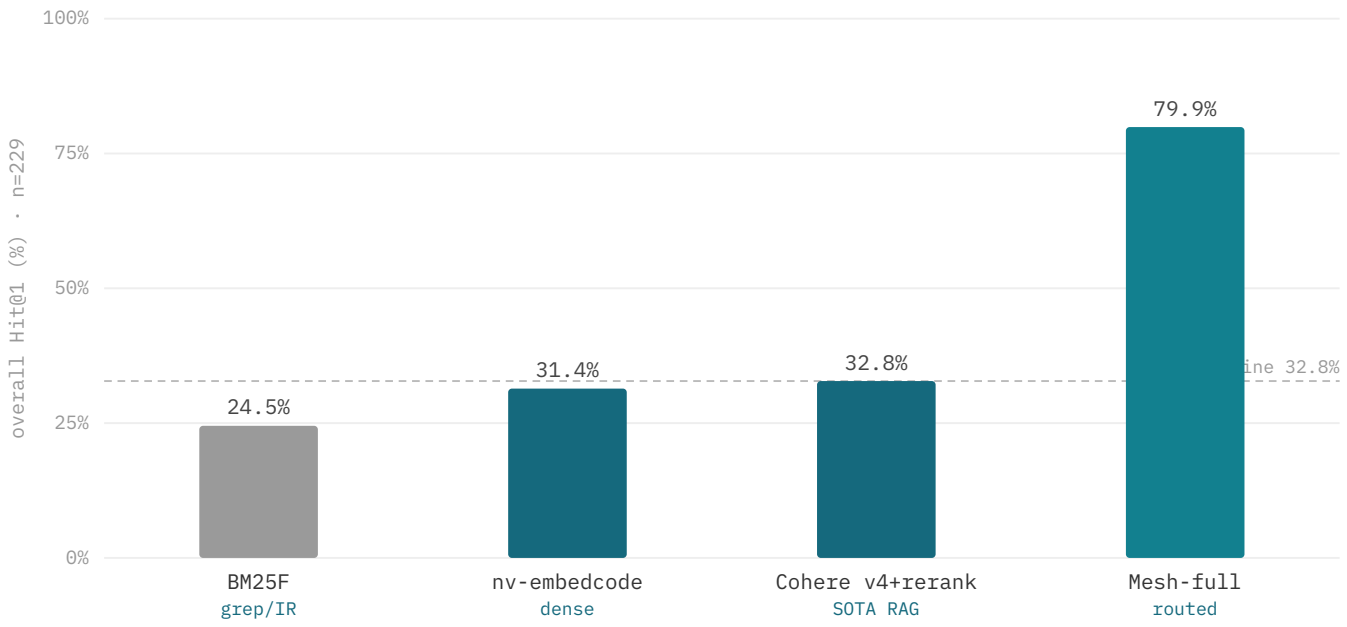


Figure §12.9. Overall Hit@1 by arm on the n=229 suite: Mesh 79.9% vs the best single-method baseline 32.8% (Cohere Embed v4 + Rerank 3.5) — §12.9.

Arm (what it represents)	Hit@1	MRR	Recall@5	exact (120)	conceptual (56)	impact (53)
BM25F — full-file lexical (grep / classic IR)	24.5%	0.375	50.2%	34.2%	17.9%	9.4%
dense — nv-embedcode-7B (SOTA code embedding)	31.4%	0.435	58.5%	20.8%	69.6%	15.1%
Cohere Embed v4 + Rerank 3.5 (SOTA managed RAG)	32.8%	0.462	62.9%	34.2%	50.0%	11.3%
Mesh-full (intent-routed)	**79.9%**	**0.864**	**95.2%**	82.5%	69.6%	84.9%

Table 6. Hit@1 per arm and class, identical pool/queries/gold (n=229); arms and routing are described in §12.9. Reproducible from `apps/cli/bench/cc-vs-mesh/final-bench.mjs`.

This section asks a sharper question than §12.6's token economics — *does the architecture beat the strongest retrieval methods, not just a grep agent?* — by bringing the competitors onto identical pool, queries, and gold, swapping only the retrieval method. The result is decisive: **Mesh wins overall Hit@1 79.9% vs 32.8% for the best single-method baseline — 2.4×, ≥ every baseline on every class**, at **McNemar p<0.001** against all three (bootstrap 95% CI on the Hit@1 delta far from zero: vs Cohere [+39.7, +54.6], vs dense [+41.0, +55.5], vs BM25 [+48.5, +62.9]; discordant pairs 117:9 vs Cohere; **Cohen's h = 0.994 vs Cohere, very large effect**). MRR follows the same ordering: Mesh 0.864 vs Cohere 0.462 / dense 0.435 / BM25 0.375. Recall@5 is Mesh 95.2% vs 62.9% / 58.5% / 50.2%; Recall@10 is 97.4% vs 70.7% / 67.2% / 63.3%. The win comes from two capabilities embedding- and grep-based retrieval structurally lack: **symbol-aware exact lookup** (82.5% vs ≤34.2%) and **dependency-graph impact** (84.9% vs ≤15.1%, 5.6×).

The comparison is file-level over the full repository file set, distinct from the chunk-level production numbers [N1]; it is additive evidence that the routed architecture beats SOTA single-method retrieval — no baseline leads it on any class.

12.10 Same-Model Scaffold Comparison: Agentic Localization

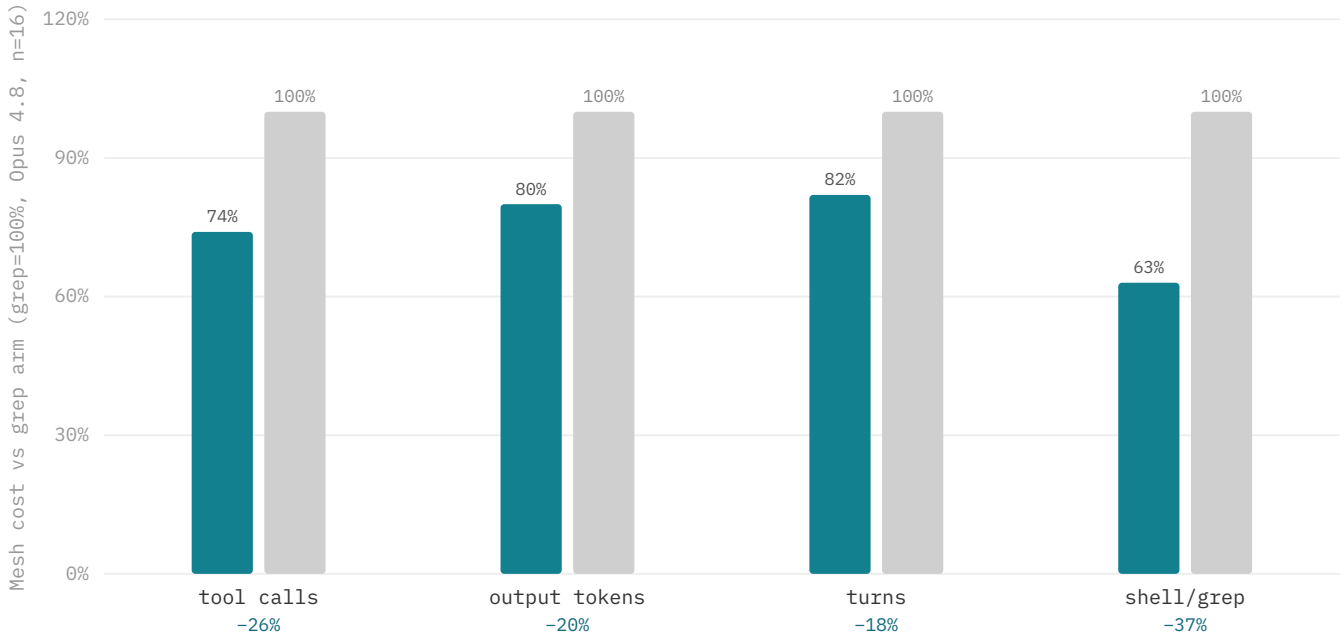


Figure §12.10. Same model (Opus 4.8) in both arms, n=16 LocBench issue-localization tasks: Mesh reaches the gold file in fewer agent steps and tokens than a grep agent at equal recall (Acc@5 100% both). [N6]

arm · model	Acc@1	Acc@5	tool calls /inst	output tok /inst
grep agent · Opus 4.8	93.8%	100%	8.8	1,614
Mesh · Opus 4.8	87.5%	100%	**6.5**	**1,299**
grep agent · Haiku 4.5	93.8%	100%	21.4	3,120
Mesh · Haiku 4.5	93.8%	100%	16.4	2,432

Table 7. LocBench file localization, same LLM in both arms (n=16); only the retrieval scaffold differs. Acc@5 is tied at 100% throughout; the Opus Acc@1 difference is a single instance (McNemar p=1.0, not significant — noise in either direction at n=16). Per-arm cost counted from agent transcripts. [N6]

Where §12.3 and §12.9 isolate the retriever, this isolates the *scaffold*. On 16 LocBench GitHub-issue localization tasks the same model runs twice — once with ripgrep and file reads only, once with Mesh's routed `semantic_search` — so the only variable is how evidence is found. Final-file accuracy is a tie by construction: a frontier agent loop saturates localization (Acc@5 100% in every arm; Acc@1 within one instance, McNemar p=1.0). The separation is in **cost**: Mesh reaches the same answer with **−26% tool calls and −20% output tokens at Opus 4.8**, and **−23% / −22% at Haiku 4.5** — the edge holds across model tiers, so it is not an artifact of a weak model leaning on retrieval to compensate [N6].

The mechanism cuts both ways, and we report it plainly. Mesh converges quickly by trusting the ranking; when the top candidate is wrong but the gold sits lower in the shortlist, the agent can stop early — the single Acc@1 the Mesh arm concedes at Opus (the gold was within its top-5). A grep agent's wider search occasionally recovers such a case, at the cost of more steps. This is an efficiency-for-robustness trade *at equal recall*, and it is the cost thesis in miniature: once models are strong enough to localize either way, the architecture's value is fewer steps and tokens to the same place, not a higher ceiling.

18 Conclusion

Mesh contributes an integrated, ablatable, cost-normalized architecture for repository-scale code agents, paired with an evaluation contract that ties each result to a specific layer. The pre-inference control plane — representation → evidence → context → economics → runtime — decomposes the agent loop into env-strippable layers with explicit token budgets, confidence gating, and recovery policies, treating context as a managed resource assembled before every turn rather than as an unbounded side effect of larger windows. On a grounded 229-task suite across six external repositories, intent-routed Mesh reaches Hit@1 79.9% / Hit@8 96.9% and, in a controlled head-to-head on identical data, beats the strongest retrieval baselines (full-file BM25, a SOTA code embedding, and SOTA managed retrieve-and-rerank) 2.4× overall at McNemar $p<0.001$ (§12.9) — while, on token economics against a competent grep-based agentic searcher, localizing at −69.7% tokens per query and −55.5% over multi-turn sessions, winning tokens on every repository. These are repo-grounded results; the lasting argument is that cost-normalized agents — where every token and dollar is accounted against task success — should be a first-class design primitive, and that a pre-inference control plane is a practical way to enforce it.